

CHRONO CARD.

A little strategy. A little borrowed time.
A dungeon crawler where every move is a card.

Draw a hand. Read the enemy's next move. Spend two plays to change your fate. Escape five rooms—alone, beside a friend, or in a live party of four.

The idea

Replace reflexes with deliberate choices. Movement, combat, and teamwork all come from the same hand. Every enemy announces its exact attack tiles before the party acts, so a risky move is an informed decision.

The experience

A compact pixel dungeon and a parchment card hand keep the next decision visible. Click a card, then a highlighted tile. Separate HP pools and alternating turns make co-op easy to follow.

What is playable

Core expedition	Five authored rooms; ten card types; turret, patroller, and chaser enemies; telegraphed attacks; shields, hazards, healing, win and loss.
Three ways to play	Offline solo, local couch duo, and authoritative online co-op for 2–4 players with codes, invite links, and a 90-second reconnect window.
Extra features	View-only spectator links and a UTC daily seed challenge. The server records completed runs to a persistent SQLite leaderboard.

Built to stay in sync

TypeScript workspaces share one deterministic rules engine between the Phaser 3 client and the Node/Express + Socket.IO server. Online clients send intentions; the server validates and resolves them. Redis snapshots restore rooms, while SQLite keeps completed daily scores.

Delivery: runnable source, tests, deployment configuration, and this two-page PDF. Public hosting is deferred by the owner. Local start: `npm ci && npm run dev`; open `localhost:5173`.

From brief to playable.

This log records the actual instructions visible in the implementation session. Development decisions below summarize the assistant's work; they are not invented user prompts or claims of human playtesting.

01 · USER REQUEST

“read prd and timeline and complete all work.”

Inputs: the Chrono Card product requirements and day-by-day timeline. The repository initially contained only its README.

02 · IMPLEMENTATION DECISIONS

Selected separate HP, alternating turns, five rooms, and a shared deterministic engine. Built the card set and fixed enemy telegraphs first, then the Phaser/DOM interface, authoritative networking, reconnect handling, persistence, and extra features.

03 · ASSISTANT CLARIFICATION

Asked whether an existing Render/Railway server project and Vercel/Netlify client project should be used for the PRD's public hosting deliverable, while continuing local implementation.

04 · USER DIRECTION

“will deploy it later”

Public deployment and hosted smoke testing were deferred. Render, Vercel, and Docker configuration plus an explicit deployment checklist are included for the later launch.

How the work was checked

- Card effects, legal targets, seeded replay, turn order, and all five rooms.
- Legal solo, duo, and four-player full-run simulations.
- Real Socket.IO party completion, spectators, private tokens, stale actions, and reconnects.
- Chromium checks for visible gameplay, refresh, mobile layout, and keyboard controls.

AI contribution and boundaries

The assistant authored the implementation, original procedural pixel/SVG artwork, tests, and handoff documents. The frontend design skill guided the visual hierarchy and readable telegraphs.

Optional ghost allies and cosmetic unlocks remain outside launch scope. Cloud hosting was not performed. No public link is represented as live.

Reproducible handoff: `npm run build`, `npm test`, and `npm run test:e2e`. This PDF is generated from editable HTML with `npm run docs:pdf`. Add the final hosted links before submission.